

Router

Router層の責務

Router層は、EchoのAPI path、HTTP method、Middleware、Controllerを接続する。

Routerは業務判断を行わない。

Routerは、どのURLにどのMiddlewareを適用し、どのController関数へ流すかだけを定義。

項目	Routerでやる	Routerでやらない
Path登録	<code>GET /beans</code> などのAPI pathを登録する	業務判断はしない
Method登録	GET/POST/PUT/PATCH/DELETEを指定する	Request bodyの中身を検証しない
Middleware接続	Auth、Admin、CSRF、RateLimitなどを適用する	Middleware内部処理を書かない
Controller接続	Controller関数をhandlerとして登録する	Usecaseを直接呼ばない
Group化	public/auth/user/adminなどでルートを分ける	権限の中身を判定しない
Health登録	<code>/health</code> , <code>/ready</code> を登録する	DB接続確認ロジックを直接書かない

Router共通方針

方針	内容
Routerは薄くする	API pathとMiddleware適用だけを書く
Controllerだけ呼ぶ	RouterからUsecaseやRepositoryを呼ばない
Middleware順序を固定する	Auth、Admin、RateLimit、CSRFの順番を崩さない
業務判断を書かない	保存可否、評価可否、ランキング、モーダル判断を書かない
Groupを分ける	public, auth, user, admin, guest/actor系で整理する
OpenAPIとズラさない	Router登録とOpenAPI pathを一致させる

Router関数一覧

関数名	役割
<code>NewRouter</code>	Echo routerを作成し、全Middlewareと全Routeを登録する
<code>RegisterGlobalMiddlewares</code>	全API共通Middlewareを登録する
<code>RegisterHealthRoutes</code>	<code>/health</code> , <code>/ready</code> を登録する
<code>RegisterPublicRoutes</code>	認証不要のBean/Article/CSRFを登録する
<code>RegisterAuthRoutes</code>	signup/login/refresh/logout/meを登録する
<code>RegisterUserRoutes</code>	保存・評価などUser必須ルートを登録する
<code>RegisterActorRoutes</code>	UserまたはGuestで使えるevent/recommendation/modal/searchを登録する <code>/search/beans</code> と <code>/search/articles</code> を含める
<code>RegisterAdminRoutes</code>	Admin専用ルートを登録する

Router依存関係

RouterはControllerとMiddlewareを受け取る。

RouterはUsecase、Repository、DB、Redisを直接受け取らない。

```
type RouterDeps struct {
    Controllers ControllerDeps
    Middlewares  MiddlewareDeps
}
```

```

type ControllerDeps struct {
    Auth          *controller.AuthController
    CSRF          *controller.CSRFController
    Bean          *controller.BeanController
    Article       *controller.ArticleController
    Event         *controller.EventController
    Recommendation *controller.RecommendationController
    Modal         *controller.ModalController
    Save          *controller.SaveController
    Search        *controller.SearchController
    Rating        *controller.RatingController
    AdminBean     *controller.AdminBeanController
    AdminArticle  *controller.AdminArticleController
    AdminRelation *controller.AdminRelationController
    AdminBatch    *controller.AdminBatchController
    AdminAudit    *controller.AdminAuditController
    Health        *controller.HealthController
}

type MiddlewareDeps struct {
    Recover          echo.MiddlewareFunc
    RequestID        echo.MiddlewareFunc
    SecureHeaders    echo.MiddlewareFunc
    CORS             echo.MiddlewareFunc
    BodyLimit        echo.MiddlewareFunc
    AccessLog        echo.MiddlewareFunc

    Auth            echo.MiddlewareFunc
    OptionalAuth    echo.MiddlewareFunc
    Admin           echo.MiddlewareFunc
    GuestSession    echo.MiddlewareFunc
    Actor           echo.MiddlewareFunc
    CSRF            echo.MiddlewareFunc

    RateLimitPublicRead    echo.MiddlewareFunc
    RateLimitAuthIP        echo.MiddlewareFunc
    RateLimitRefresh       echo.MiddlewareFunc
    RateLimitUser          echo.MiddlewareFunc
    RateLimitSearch        echo.MiddlewareFunc
    RateLimitEvent         echo.MiddlewareFunc
    RateLimitRecommendation echo.MiddlewareFunc
    RateLimitModal         echo.MiddlewareFunc
    RateLimitAdmin         echo.MiddlewareFunc
    RateLimitAdminBatch    echo.MiddlewareFunc
}

```

全体共通Middleware

全APIに適用する。

順番	Middleware	理由
1	RecoverMiddleware	panicを500で返すため
2	RequestIDMiddleware	request_idを後続処理へ渡すため
3	SecureHeadersMiddleware	安全ヘッダーを付与するため
4	CORSMiddleware	Frontend originを制限するため
5	BodyLimitMiddleware	巨大Bodyを拒否するため

順番	Middleware	理由
6	<code>AccessLogMiddleware</code>	request/responseログを残すため

Health Routes

HTTP	Path	Middleware	Controller	役割
GET	<code>/health</code>	なし	<code>HealthController.Health</code>	アプリの生存確認
GET	<code>/ready</code>	なし	<code>HealthController.Ready</code>	DB/Redis接続確認

Public Routes

Public Routesは、認証不要で使えるAPI。

`/beans` と `/articles` は、公開済みコンテンツの一覧を返すAPIである。

キーワード、味覚条件、焙煎度、産地などを指定する条件検索はPublic Routesでは扱わない。

条件検索は、UserまたはGuestSessionを持つActor向けAPIとして、Actor Routesの `/search/beans` と `/search/articles` に分離する。

HTTP	Path	Middleware	Controller	役割
GET	<code>/beans</code>	<code>RateLimitPublicRead</code>	<code>BeanController.ListBeans</code>	Bean一覧
GET	<code>/beans/:id</code>	<code>RateLimitPublicRead</code>	<code>BeanController.GetBeanDetail</code>	Bean詳細
GET	<code>/articles</code>	<code>RateLimitPublicRead</code>	<code>ArticleController.ListArticles</code>	Article一覧
GET	<code>/articles/:slug/preview</code>	<code>RateLimitPublicRead</code>	<code>ArticleController.GetArticlePreview</code>	Guest向けArticle preview
GET	<code>/auth/csrf</code>	<code>RateLimitPublicRead</code>	<code>CSRFController.Issue</code>	CSRF token発行

Auth Routes

Auth Routesは、signup/login/refresh/logout/logout-all/meを扱う。

RefreshはAccessTokenが期限切れでも使うため、AuthMiddlewareを使わない。

LogoutとLogoutAllは認証済みUser操作なので、Auth → RateLimit → CSRFの順番にする。

HTTP	Path	Middleware	Controller	役割
POST	<code>/auth/signup</code>	<code>RateLimitAuthIP</code>	<code>AuthController.Signup</code>	会員登録
POST	<code>/auth/login</code>	<code>RateLimitAuthIP</code>	<code>AuthController.Login</code>	ログイン
POST	<code>/auth/refresh</code>	<code>RateLimitRefresh</code> → <code>CSRFMiddleware</code>	<code>AuthController.Refresh</code>	AccessToken再発行
POST	<code>/auth/logout</code>	<code>AuthMiddleware</code> → <code>RateLimitUser</code> → <code>CSRFMiddleware</code>	<code>AuthController.Logout</code>	現在のRefreshToken familyを失効してログアウト
POST	<code>/auth/logout-all</code>	<code>AuthMiddleware</code> → <code>RateLimitUser</code> → <code>CSRFMiddleware</code>	<code>AuthController.LogoutAll</code>	全端末ログアウト
GET	<code>/auth/me</code>	<code>AuthMiddleware</code> → <code>RateLimitUser</code>	<code>AuthController.Me</code>	ログイン中ユーザー取得

User Routes

User Routesは、ログイン済みUserのみ使えるAPI。

Guestは保存・評価できない。

HTTP	Path	Middleware	Controller	役割
GET	<code>/articles/:slug</code>	<code>AuthMiddleware</code> → <code>RateLimitUser</code>	<code>ArticleController.GetArticleDetail</code>	User向けArticle全文
GET	<code>/saved</code>	<code>AuthMiddleware</code> → <code>RateLimitUser</code>	<code>SaveController.ListSavedContents</code>	保存一覧
POST	<code>/saved</code>	<code>AuthMiddleware</code> → <code>RateLimitUser</code>	<code>SaveController.SaveContent</code>	保存

HTTP	Path	Middleware	Controller	役割
DELETE	/saved	AuthMiddleware → RateLimitUser	SaveController.UnsaveContent	保存解除
POST	/ratings	AuthMiddleware → RateLimitUser	RatingController.RateContent	評価
GET	/ratings/me	AuthMiddleware → RateLimitUser	RatingController.GetMyRating	自分の評価取得

Actor Routes

Actor Routesは、UserまたはGuestSessionを持つAPIを登録する。

検索APIは参照系GETとして扱い、検索結果だけを返す。

行動ログはPOST /eventsで記録する。

re_searchもPOST /eventsで記録し、CSRF対象にする。

HTTP	Path	Middleware	Controller	役割
GET	/search/beans	OptionalAuth → GuestSession → Actor → RateLimitSearch	SearchController.SearchBeans	Bean検索結果を返す
GET	/search/articles	OptionalAuth → GuestSession → Actor → RateLimitSearch	SearchController.SearchArticles	Article検索結果を返す
POST	/events	OptionalAuth → GuestSession → Actor → RateLimitEvent → CSRF	EventController.RecordEvent	content_view, impression, stay, click, re_search を記録する

Event

HTTP	Path	Middleware	Controller	役割
POST	/events	OptionalAuthMiddleware → GuestSessionMiddleware → ActorMiddleware → RateLimitEvent → CSRFMiddleware	EventController.RecordEvent	行動ログ記録

Modal

HTTP	Path	Middleware	Controller	役割
GET	/recommendation-modal	OptionalAuthMiddleware → GuestSessionMiddleware → ActorMiddleware → RateLimitModal	ModalController.GetRecommendationModal	推薦モーダル取得
POST	/recommendation-modal/:id/click	OptionalAuthMiddleware → GuestSessionMiddleware → ActorMiddleware → RateLimitModal → CSRFMiddleware	ModalController.RecordModalClick	モーダルクリック記録
POST	/recommendation-modal/:id/close	OptionalAuthMiddleware → GuestSessionMiddleware → ActorMiddleware → RateLimitModal → CSRFMiddleware	ModalController.RecordModalClose	モーダル閉じる記録

Admin Routes

Admin Routesは、Adminのみ使えるAPI。

全Admin Routesは、AuthMiddleware → AdminMiddleware → RateLimit の順番で適用。

Admin Bean

HTTP	Path	Middleware	Controller	役割
POST	/admin/beans	AuthMiddleware → AdminMiddleware → RateLimitAdmin	AdminBeanController.CreateBean	Bean作成

HTTP	Path	Middleware	Controller	役割
PUT	/admin/beans/:id	AuthMiddleware → AdminMiddleware → RateLimitAdmin	AdminBeanController.UpdateBean	Bean更新
PATCH	/admin/beans/:id/publish	AuthMiddleware → AdminMiddleware → RateLimitAdmin	AdminBeanController.PublishBean	Bean公開
PATCH	/admin/beans/:id/unpublish	AuthMiddleware → AdminMiddleware → RateLimitAdmin	AdminBeanController.UnpublishBean	Bean非公開

Admin Article

HTTP	Path	Middleware	Controller	役割
POST	/admin/articles	AuthMiddleware → AdminMiddleware → RateLimitAdmin	AdminArticleController.CreateArticle	Article作成
PUT	/admin/articles/:id	AuthMiddleware → AdminMiddleware → RateLimitAdmin	AdminArticleController.UpdateArticle	Article更新
PATCH	/admin/articles/:id/publish	AuthMiddleware → AdminMiddleware → RateLimitAdmin	AdminArticleController.PublishArticle	Article公開
PATCH	/admin/articles/:id/unpublish	AuthMiddleware → AdminMiddleware → RateLimitAdmin	AdminArticleController.UnpublishArticle	Article非公開

Admin Relation

HTTP	Path	Middleware	Controller	役割
POST	/admin/bean-articles	AuthMiddleware → AdminMiddleware → RateLimitAdmin	AdminRelationController.CreateBeanArticleRelation	BeanとArticleを 関連付ける
DELETE	/admin/bean-articles	AuthMiddleware → AdminMiddleware → RateLimitAdmin	AdminRelationController.DeleteBeanArticleRelation	BeanとArticleの 関連付け解除
PATCH	/admin/bean-articles/order	AuthMiddleware → AdminMiddleware → RateLimitAdmin	AdminRelationController.UpdateBeanArticleOrder	関連記事の表示順 更新

Admin Batch

HTTP	Path	Middleware	Controller	役割
POST	/admin/batch/ranking	AuthMiddleware → AdminMiddleware → RateLimitAdminBatch	AdminBatchController.RunRankingBatch	ランキングバッチ 手動実行

Admin Audit

HTTP	Path	Middleware	Controller	役割
GET	/admin/audit-logs	AuthMiddleware → AdminMiddleware → RateLimitAdmin	AdminAuditController.ListAuditLogs	監査ログ一覧取 得

Router実装イメージ

```
func NewRouter(e *echo.Echo, deps RouterDeps) {
    RegisterGlobalMiddlewares(e, deps.Middlewares)
```

```

    RegisterHealthRoutes(e, deps.Controllers.Health)
    RegisterPublicRoutes(e, deps.Controllers, deps.Middlewares)
    RegisterAuthRoutes(e, deps.Controllers, deps.Middlewares)
    RegisterUserRoutes(e, deps.Controllers, deps.Middlewares)
    RegisterActorRoutes(e, deps.Controllers, deps.Middlewares)
    RegisterAdminRoutes(e, deps.Controllers, deps.Middlewares)
}

```

RegisterGlobalMiddlewares

```

func RegisterGlobalMiddlewares(e *echo.Echo, mw MiddlewareDeps) {
    e.Use(mw.Recover)
    e.Use(mw.RequestID)
    e.Use(mw.SecureHeaders)
    e.Use(mw.CORS)
    e.Use(mw.BodyLimit)
    e.Use(mw.AccessLog)
}

```

RegisterHealthRoutes

```

func RegisterHealthRoutes(e *echo.Echo, h *controller.HealthController) {
    e.GET("/health", h.Health)
    e.GET("/ready", h.Ready)
}

```

RegisterPublicRoutes

```

func RegisterPublicRoutes(e *echo.Echo, c ControllerDeps, mw MiddlewareDeps) {
    e.GET("/beans", c.Bean.ListBeans, mw.RateLimitPublicRead)
    e.GET("/beans/:id", c.Bean.GetBeanDetail, mw.RateLimitPublicRead)

    e.GET("/articles", c.Article.ListArticles, mw.RateLimitPublicRead)
    e.GET("/articles/:slug/preview", c.Article.GetArticlePreview, mw.RateLimitPublicRead)

    e.GET("/auth/csrf", c.CSRF.Issue, mw.RateLimitPublicRead)
}

```

RegisterAuthRoutes

```

func RegisterAuthRoutes(e *echo.Echo, c ControllerDeps, mw MiddlewareDeps) {
    e.POST(
        "/auth/signup",
        c.Auth.Signup,
        mw.RateLimitAuthIP,
    )

    e.POST(
        "/auth/login",
        c.Auth.Login,
        mw.RateLimitAuthIP,
    )
}

```

```

    )

    e.POST(
        "/auth/refresh",
        c.Auth.Refresh,
        mw.RateLimitRefresh,
        mw.CSRF,
    )

    e.POST(
        "/auth/logout",
        c.Auth.Logout,
        mw.Auth,
        mw.RateLimitUser,
        mw.CSRF,
    )

    e.POST(
        "/auth/logout-all",
        c.Auth.LogoutAll,
        mw.Auth,
        mw.RateLimitUser,
        mw.CSRF,
    )

    e.GET(
        "/auth/me",
        c.Auth.Me,
        mw.Auth,
        mw.RateLimitUser,
    )
}

```

RegisterUserRoutes

```

func RegisterUserRoutes(e *echo.Echo, c ControllerDeps, mw MiddlewareDeps) {
    e.GET(
        "/articles/:slug",
        c.Article.GetArticleDetail,
        mw.Auth,
        mw.RateLimitUser,
    )

    e.GET(
        "/saved",
        c.Save.ListSavedContents,
        mw.Auth,
        mw.RateLimitUser,
    )

    e.POST(
        "/saved",
        c.Save.SaveContent,
        mw.Auth,
        mw.RateLimitUser,
    )
}

```

```

e.DELETE(
    "/saved",
    c.Save.UnsaveContent,
    mw.Auth,
    mw.RateLimitUser,
)

e.POST(
    "/ratings",
    c.Rating.RateContent,
    mw.Auth,
    mw.RateLimitUser,
)

e.GET(
    "/ratings/me",
    c.Rating.GetMyRating,
    mw.Auth,
    mw.RateLimitUser,
)
}

```

RegisterActorRoutes

```

func RegisterActorRoutes(e *echo.Echo, c ControllerDeps, mw MiddlewareDeps) {
    actor := []echo.MiddlewareFunc{
        mw.OptionalAuth,
        mw.GuestSession,
        mw.Actor,
    }

    e.GET(
        "/search/beans",
        c.Search.SearchBeans,
        append(actor, mw.RateLimitSearch)...,
    )

    e.GET(
        "/search/articles",
        c.Search.SearchArticles,
        append(actor, mw.RateLimitSearch)...,
    )

    e.GET(
        "/recommendations",
        c.Recommendation.ListRecommendations,
        append(actor, mw.RateLimitRecommendation)...,
    )

    e.POST(
        "/events",
        c.Event.RecordEvent,
        append(actor, mw.RateLimitEvent, mw.CSRF)...,
    )
}

```

```

e.GET(
    "/recommendation-modal",
    c.Modal.GetRecommendationModal,
    append(actor, mw.RateLimitModal)...,
)

e.POST(
    "/recommendation-modal/:id/click",
    c.Modal.RecordModalClick,
    append(actor, mw.RateLimitModal, mw.CSRF)...,
)

e.POST(
    "/recommendation-modal/:id/close",
    c.Modal.RecordModalClose,
    append(actor, mw.RateLimitModal, mw.CSRF)...,
)
}

```

RegisterAdminRoutes

```

func RegisterAdminRoutes(e *echo.Echo, c ControllerDeps, mw MiddlewareDeps) {
    admin := e.Group("/admin", mw.Auth, mw.Admin)

    admin.POST("/beans", c.AdminBean.CreateBean, mw.RateLimitAdmin)
    admin.PUT("/beans/:id", c.AdminBean.UpdateBean, mw.RateLimitAdmin)
    admin.PATCH("/beans/:id/publish", c.AdminBean.PublishBean, mw.RateLimitAdmin)
    admin.PATCH("/beans/:id/unpublish", c.AdminBean.UnpublishBean, mw.RateLimitAdmin)

    admin.POST("/articles", c.AdminArticle.CreateArticle, mw.RateLimitAdmin)
    admin.PUT("/articles/:id", c.AdminArticle.UpdateArticle, mw.RateLimitAdmin)
    admin.PATCH("/articles/:id/publish", c.AdminArticle.PublishArticle, mw.RateLimitAdmin)
    admin.PATCH("/articles/:id/unpublish", c.AdminArticle.UnpublishArticle, mw.RateLimitAdmin)

    admin.POST("/bean-articles", c.AdminRelation.CreateBeanArticleRelation, mw.RateLimitAdmin)
    admin.DELETE("/bean-articles", c.AdminRelation.DeleteBeanArticleRelation, mw.RateLimitAdmin)
    admin.PATCH("/bean-articles/order", c.AdminRelation.UpdateBeanArticleOrder, mw.RateLimitAdmin)

    admin.POST("/batch/ranking", c.AdminBatch.RunRankingBatch, mw.RateLimitAdminBatch)

    admin.GET("/audit-logs", c.AdminAudit.ListAuditLogs, mw.RateLimitAdmin)
}

```

Routerでやってはいけないこと

NG	理由
Usecaseを直接呼ぶ	Controllerの責務を飛ばすため
Repositoryを直接呼ぶ	Clean Architectureが崩れる
GORMを使う	RouterがDB実装に依存する
Redis Clientを使う	Middleware/Repositoryの責務を奪う

NG	理由
user_idをPathやBodyから信用する	なりすまし可能
保存可否を書く	SaveUsecaseの責務
評価可否を書く	RatingUsecaseの責務
ranking scoreを計算する	RankingUsecaseの責務
modal表示可否を書く	ModalUsecaseの責務
RefreshToken処理を書く	AuthUsecaseの責務
CSRF判定を書く	CSRFMiddlewareの責務
RateLimit判定を書く	RateLimitMiddlewareの責務
OpenAPIと違うPathを登録する	フロント連携・レビューで事故る